

```

disp('Welcome to the Quantum Harmonic Oscillator Wavefunction Plotter!');
disp('-----');
while true
    % Input quantum number for electron
    n_electron = input('Enter the quantum number for electron (non-negative integer), or type "exit" to quit: ', 's');
    if strcmpi(n_electron, 'exit')
        disp('Exiting program...');
        break;
    end
    n_electron = str2double(n_electron);
    % Check if the input is a non-negative integer
    if ~isnan(n_electron) && n_electron >= 0 && round(n_electron) == n_electron
        % Display the calculated formula
        disp(['Wavefunction formula for n_electron = ', num2str(n_electron), ':']);
        disp(['\psi_{electron}(x) = 1/sqrt(2^', num2str(n_electron), ' * ', 'factorial(', num2str(n_electron), ') * sqrt(pi)) * Hermite_', num2str(n_electron), '']];

        % Define parameters
        x_max = 5; % Maximum x value for plotting
        num_points = 100; % Number of points for plotting
        % Define x values
        x = linspace(-x_max, x_max, num_points);
        % Calculate wavefunction for electron using Hermite polynomial
        Hn_electron = hermiteH(n_electron, x);
        psi_electron = 1/sqrt(2^n_electron * factorial(n_electron) * sqrt(pi)) .* Hn_electron .* exp(-x.^2/2);

        % Calculate probability density
        probability_density_electron = abs(psi_electron).^2;
        % Calculate wavefunction for proton using different quantum number
        n_proton = n_electron + 1; % Using a different quantum number for proton
        Hn_proton = hermiteH(n_proton, x);
        psi_proton = 1/sqrt(2^n_proton * factorial(n_proton) * sqrt(pi)) .* Hn_proton .* exp(-x.^2/2);

        % Calculate probability density
        probability_density_proton = abs(psi_proton).^2;
        % Create a figure and axis for animation
        figure;
        ax1 = subplot(2,1,1);
        line_handle_electron = plot(ax1, x, psi_electron, 'LineWidth', 1.5, 'Color', 'b');
        hold on;
        plot(ax1, x, probability_density_electron, '--', 'LineWidth', 1.5, 'Color', 'b'); % Plot probability density
        line_handle_proton = plot(ax1, x, psi_proton, 'LineWidth', 1.5, 'Color', 'r');
        plot(ax1, x, probability_density_proton, '--', 'LineWidth', 1.5, 'Color', 'r'); % Plot probability density
        xlabel(ax1, 'x');
        ylabel(ax1, '\psi(x), |\psi(x)|^2');
        title(ax1, ['Wavefunction and Probability Density of Quantum Harmonic Oscillator for n_electron = ', num2str(n_electron), ', n_proton = ', num2str(n_proton)]);
        legend(ax1, {'\psi_{electron}(x)', '|\psi_{electron}(x)|^2', '\psi_{proton}(x)', '|\psi_{proton}(x)|^2'});
        grid(ax1, 'on');
        % Plot energy levels for electron
        energies_electron = (0:n_electron) + 0.5;
        ax2 = subplot(2,1,2);
        for i = 1:length(energies_electron)
            if i == n_electron + 1 % Plot only the energy level corresponding to the entered quantum number
                line([x(1), x(end)], [energies_electron(i), energies_electron(i)], 'Color', 'b', 'LineStyle', '--', 'Parent', ax2);
                hold on;
            end
        end
    end
    % Plot energy levels for proton
    energies_proton = (0:n_proton) + 0.5;
    for i = 1:length(energies_proton)
        if i == n_proton + 1 % Plot only the energy level corresponding to the quantum number of the proton
            line([x(1), x(end)], [energies_proton(i), energies_proton(i)], 'Color', 'r', 'LineStyle', '--', 'Parent', ax2);
            hold on;
        end
    end
end

% Animation loop
t = 0;
dt = 0.09; % Decreased pause duration for smoother animation
while true
    % Update the wavefunctions with time-dependent phase factor
    psi_electron_t = psi_electron .* exp(-1i * t);
    psi_proton_t = psi_proton .* exp(-1i * t);
    set(line_handle_electron, 'YData', real(psi_electron_t));
    set(line_handle_proton, 'YData', real(psi_proton_t));
    drawnow;
    pause(dt); % Decreased pause duration

    % Increase time
    t = t + dt;

    % Check if user wants to stop animation
    stop = input('Press any key to stop animation: ');
    if stop == 'q' || stop == 'Q' || stop == 'quit' || stop == 'Quit'
        break;
    end
end

```

```

        if isempty(findobj('type','figure'))
            break;
        end
    end
end
% Allow the user to adjust the range of the plot
x_max = input('Enter the maximum x value for plotting (press Enter to continue with default): ');
if ~isempty(x_max)
    x = linspace(-x_max, x_max, num_points);
    % Recalculate wavefunctions and probability densities
    Hn_electron = hermiteH(n_electron, x);
    psi_electron = 1/sqrt(2^n_electron * factorial(n_electron) * sqrt(pi)) .* Hn_electron .* exp(-x.^2/2);
    probability_density_electron = abs(psi_electron).^2;
    Hn_proton = hermiteH(n_proton, x);
    psi_proton = 1/sqrt(2^n_proton * factorial(n_proton) * sqrt(pi)) .* Hn_proton .* exp(-x.^2/2);
    probability_density_proton = abs(psi_proton).^2;
    % Update plots
    set(line_handle_electron, 'XData', x, 'YData', psi_electron);
    plot(ax1, x, probability_density_electron, '--', 'LineWidth', 1.5, 'Color', 'b');
    set(line_handle_proton, 'XData', x, 'YData', psi_proton);
    plot(ax1, x, probability_density_proton, '--', 'LineWidth', 1.5, 'Color', 'r');
    xlabel(ax1, 'x');
    ylabel(ax1, '\psi(x), |\psi(x)|^2');
    title(ax1, ['Wavefunction and Probability Density of Quantum Harmonic Oscillator for n_electron = ', num2str(n_electron), ', n_proton = ', num2str(n_proton)]);
    legend(ax1, {'\psi_{electron}(x)', '|\psi_{electron}(x)|^2', '\psi_{proton}(x)', '|\psi_{proton}(x)|^2'});
    grid(ax1, 'on');
end
input('Press Enter to continue...');
close;
else
    disp('Please enter a valid non-negative integer or type "exit" to quit. ');
    continue; % Redirect the user to input the next quantum number
end
end
end

```